

Ain Shams University - Faculty of Engineering
CSE336 Software Design Patterns - Fall 2026

Software Requirements Specification

Hope Bridge

Non-Profit Organization Management System

Major Task - Phase 1 Submission

Team Members

Name	Student ID
Saifeldin Tamer ElSayes	22P0233
Mohamed Emad Eldeen	22P0065
Ahmed Khaled Hamdy	22P0024
Yousef Sameh Fathi	22P0010
Seif Eldeen Ibrahim	22P0023
Mohamed Khaled Mohamed Anwar	22P0041

Version 1.0 - November 2026

Contents

1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Definitions and Abbreviations 3
1.4 References 3
1.5 Document Overview 4
2. Overall Description 4
2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Classes 4
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 5
2.6 Assumptions and Dependencies 5
3. Functional Requirements 6
3.1 People and Accounts 6
3.2 Donation Management 6
3.3 Event Management 6
3.4 Volunteer Management 7
3.5 Beneficiary Management 7
3.6 Communication 7
3.7 Administration - Lookup Tables 8
4. Client Requirements 9
5. Non-Functional Requirements 9
6. System Design 11
6.1 Architecture - Model View Controller 11
6.2 Class Diagrams 12
6.3 Design Patterns Applied 20
6.4 CRUD Lookup Tables 22
6.5 Main Design Decisions 23
7. Work Distribution 24
8. Traceability to the Phase 1 Grading Criteria 25
9. Running the System 25
9.1 Build and Run 25
9.2 Sample Data 25
9.3 Demonstration Scenario 26

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes **Hope Bridge**, a management system for a non-profit organization. It defines what the system must do, the quality attributes it must have, and how its object-oriented design uses design patterns to stay flexible, maintainable and reusable.

The document is written for the course instructor and teaching assistants who evaluate the project, and for the six team members who build it. It covers Phase 1 of the CSE336 major task. The class diagrams and the Java code described here are the same design: every class in the diagrams exists in the code with the same attributes, operations and relationships.

1.2 Scope

Hope Bridge supports the daily work of a non-profit: managing people and accounts, receiving donations, organizing events and volunteers, supporting beneficiaries and communicating with all of them. The project is delivered in two phases:

- **Phase 1 (this document):** the complete class design and a working Java implementation of all five modules using the patterns taught so far - **Strategy, Observer, Decorator, Factory Method and Singleton** - organized with the **MVC** architecture, plus editable **CRUD lookup tables**.
- **Phase 2:** the remaining patterns (Command, Adapter, Facade, Template Method, Iterator, State, Proxy), admin pages, persistence to a real database and integration with real payment, e-mail and SMS providers.

Requirements that belong to Phase 2 are still listed in Section 3 so the design stays complete, and each one is marked with its phase.

1.3 Definitions and Abbreviations

Term	Meaning
NPO	Non-profit organization
Donor	A person or company that gives money (monetary donation) or goods (in-kind donation)
Beneficiary	A person, with their household, whom the organization supports
Campaign	A fundraising drive with a money goal, e.g. "Winter Blankets 2026"
Assignment	A volunteer working at one event in a given role
Distribution	A quantity of a resource given to a beneficiary
Lookup table	A small reference table (skills, categories) that admins edit at runtime
CRUD	Create, Read, Update, Delete
DAO	Data Access Object - the class that stores and retrieves one kind of record
MVC	Model-View-Controller architecture
EGP	Egyptian Pound, the currency used for all amounts

1.4 References

- CSE336 Major Assessment Task, Part 1 - project brief and evaluation criteria.
- E. Freeman, E. Robson, B. Bates, K. Sierra, Head First Design Patterns, O'Reilly.
- E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, 1995.
- CSE336 lecture slides (Strategy, Observer, Decorator, Factory, Singleton) and Labs 1, 3 and 4.

1.5 Document Overview

Section 2 describes the product and its users. Section 3 lists the functional requirements per module and Section 4 the client requirements that extend the basic brief. Section 5 gives the non-functional requirements. Section 6 presents the design: architecture, class diagrams, each design pattern, the lookup tables and the main design decisions. Section 7 shows how the work is divided in the team, Section 8 maps every Phase 1 grading item to the diagram and code that satisfies it, and Section 9 explains how to run the system.

2. Overall Description

2.1 Product Perspective

Hope Bridge is a new, self-contained system. In Phase 1 it is a Java console application that keeps its data in memory and is loaded with sample data at start-up, so every feature can be demonstrated immediately. Payments and message delivery are simulated: no real card gateway, bank, e-mail server or SMS provider is contacted, and delivered messages are placed in each person's in-app inbox.

2.2 Product Functions

Module	Main functions
People and accounts	Sign up as donor, volunteer or beneficiary; log in with a hashed password; search, update and delete people; choose a preferred contact channel
Donation management	Record money and in-kind donations, choose a payment method, link donations to campaigns, issue receipts, thank donors, produce yearly tax statements
Event management	Create fundraisers, workshops and outreach programs; register attendees; sell tickets with extras; track attendance; reschedule, move, cancel, remind and complete events
Volunteer management	Store skills, availability and preferences; suggest qualified volunteers; assign them with a role; track hours; issue certificates
Beneficiary management	Store households and their needs, distribute resources from stock, evaluate the impact of programs
Communication	Notify people automatically through their preferred channel (e-mail, SMS, social media) and keep an activity log
Administration	Maintain the lookup tables: volunteer skills, donation item categories and resource categories

2.3 User Classes

User	Description
Administrator	Maintains lookup tables and reviews the activity log
Staff member	Records donations and distributions, manages events and volunteer assignments
Donor	Gives money or goods, buys event tickets, receives receipts, statements and thank-you messages
Volunteer	Gives time at events, receives event updates and certificates
Beneficiary	Receives support and updates about events that concern them

2.4 Operating Environment

Java 17 or later (JDK) built with Apache Maven, on Windows, Linux or macOS. The user interface is the console; the project opens directly in NetBeans or IntelliJ as a Maven project.

2.5 Design and Implementation Constraints

- The design must apply the design patterns taught in the course and follow object-oriented principles: encapsulation, program to an interface, favour composition over inheritance, and the open/closed principle.
- Only the View layer may read input or print output. Model and controller classes never print.
- Phase 1 stores data in memory behind one data-access point (DatabaseManager), so Phase 2 can move to a real database without changing the rest of the system.

2.6 Assumptions and Dependencies

- All amounts are in Egyptian Pounds (EGP).
- Online and check payments are simulated in Phase 1; integration with a payment provider is Phase 2.
- E-mail, SMS and social-media delivery are simulated by placing the message in the person's inbox with its channel; real providers are Phase 2.

3. Functional Requirements

Each requirement has an identifier, a description and the phase in which it is delivered.

3.1 People and Accounts

ID	Requirement	Status
FR-P1	The system shall let a person sign up as a donor, volunteer or beneficiary with name, e-mail, phone and password.	Phase 1 - implemented
FR-P2	The system shall reject a sign-up with an e-mail already in use or a password shorter than 6 characters.	Phase 1 - implemented
FR-P3	The system shall store only a SHA-256 hash of each password and log people in by e-mail and password.	Phase 1 - implemented
FR-P4	The system shall let staff search people by name and update contact details and the preferred channel.	Phase 1 - implemented
FR-P5	The system shall refuse to delete a person who has donations, assignments, distributions or event registrations.	Phase 1 - implemented

3.2 Donation Management

ID	Requirement	Status
FR-D1	The system shall store donor details: donor type (individual or corporate), tax id and total donated.	Phase 1 - implemented
FR-D2	The system shall record monetary donations with an amount and a payment method: online card, check or cash.	Phase 1 - implemented
FR-D3	The system shall record in-kind donations with item description, quantity, estimated value and item category.	Phase 1 - implemented
FR-D4	The system shall allow new payment methods to be added without changing existing donation code.	Phase 1 - implemented
FR-D5	The system shall validate every donation and reject invalid ones (e.g. amount <= 0, missing category).	Phase 1 - implemented
FR-D6	The system shall issue a signed receipt for every recorded donation, with optional tax information and thank-you note, and re-issue it on request.	Phase 1 - implemented
FR-D7	The system shall produce a yearly donation statement for a donor's tax records.	Phase 1 - implemented
FR-D8	The system shall link each donation to a campaign and keep campaign progress and organization totals up to date.	Phase 1 - implemented
FR-D9	The system shall connect to a real payment gateway and bank clearing service.	Phase 2

3.3 Event Management

ID	Requirement	Status
FR-E1	The system shall create fundraisers (target amount), workshops (trainer, topic) and outreach programs (location, team size) with name, date, venue, capacity, ticket price and an optional required skill.	Phase 1 - implemented
FR-E2	The system shall register people while seats remain and refuse full, cancelled or completed events.	Phase 1 - implemented
FR-E3	The system shall sell tickets with optional VIP seating and dinner, each adding to the price.	Phase 1 - implemented

ID	Requirement	Status
FR-E4	The system shall mark the attendance of registered people.	Phase 1 - implemented
FR-E5	The system shall change an event's date or venue, cancel it, send reminders and complete it, informing every attendee and volunteer.	Phase 1 - implemented
FR-E6	The system shall print tickets with a QR code for entry.	Phase 2

3.4 Volunteer Management

ID	Requirement	Status
FR-V1	The system shall store volunteers' skills (from the Skill lookup table), availability and preferred event types.	Phase 1 - implemented
FR-V2	The system shall suggest qualified volunteers for an event, listing those who prefer that event type first.	Phase 1 - implemented
FR-V3	The system shall assign a volunteer to an event with a role only if the volunteer has the event's required skill.	Phase 1 - implemented
FR-V4	The system shall track hours per assignment and per volunteer, crediting them when an event is completed.	Phase 1 - implemented
FR-V5	The system shall issue a signed volunteer certificate with the total hours.	Phase 1 - implemented
FR-V6	The system shall let volunteers book their own shifts from a calendar.	Phase 2

3.5 Beneficiary Management

ID	Requirement	Status
FR-B1	The system shall store beneficiaries with address, family size and a list of prioritized needs.	Phase 1 - implemented
FR-B2	The system shall record the distribution of resources to beneficiaries and reduce the stock, refusing quantities above the stock.	Phase 1 - implemented
FR-B3	The system shall restock resources and list stock levels.	Phase 1 - implemented
FR-B4	The system shall evaluate the impact of a program with a method suited to that program (food aid or education).	Phase 1 - implemented
FR-B5	The system shall produce periodic impact reports for funders.	Phase 2

3.6 Communication

ID	Requirement	Status
FR-C1	The system shall notify everyone concerned automatically when a donation is recorded or an event changes.	Phase 1 - implemented
FR-C2	The system shall deliver each notification through the recipient's preferred channel - e-mail, SMS or social media - falling back to e-mail when needed.	Phase 1 - implemented
FR-C3	The system shall keep a notification inbox per person with read and unread messages.	Phase 1 - implemented
FR-C4	The system shall keep an activity log of every published change for administrators.	Phase 1 - implemented
FR-C5	The system shall publish public campaign posts on social-media pages.	Phase 2

3.7 Administration - Lookup Tables

ID	Requirement	Status
FR-L1	The administrator shall create, list, edit and delete volunteer skills.	Phase 1 - implemented
FR-L2	The administrator shall create, list, edit and delete donation item categories.	Phase 1 - implemented
FR-L3	The administrator shall create, list, edit and delete resource categories.	Phase 1 - implemented
FR-L4	The system shall refuse to delete a lookup row that is still used by an event, volunteer, donation, resource or need.	Phase 1 - implemented

4. Client Requirements

The brief lists the basic modules. The requirements below extend it with what the organization's staff, donors, volunteers and beneficiaries need on top of those modules. Each one is implemented in Phase 1 and traced to the classes that satisfy it.

ID	Client requirement	Satisfied by
CR-01	Every person chooses how they want to be contacted - e-mail, SMS or social media - and messages arrive through that channel.	Strategy: NotificationService + Email/Sms/SocialMediaStrategy
CR-02	A donor chooses how to pay - online card, check or cash. Cash was not in the original brief.	Strategy: PaymentStrategy + Online/Check/CashPayment
CR-03	Card numbers are never shown in full; checks show their clearing date.	OnlinePayment, CheckPayment
CR-04	A donor decides whether the receipt includes tax information and a thank-you note; the organization's signature is always added.	Decorator: TaxInfo/ThankYouNote/SignatureDecorator
CR-05	Every donor is thanked automatically right after donating.	Observer: ThankYouObserver
CR-06	Corporate and individual donors get a yearly statement for their tax records.	DonationService.generateTaxStatement()
CR-07	Event tickets can include VIP seating and dinner in any combination.	Decorator: VipDecorator, MealDecorator
CR-08	An event can never be over-booked, and nobody can register for a cancelled or completed event.	Event.register()
CR-09	A volunteer is only assigned to an event that needs a skill they actually have, and volunteers who like that type of event are suggested first.	VolunteerService.assign(), suggestVolunteers()
CR-10	Distributions can never exceed the stock, and the stock is reduced automatically.	DistributionService, Observer: StockObserver
CR-11	Food aid is measured per person fed; education by how well distributions match recorded needs.	Strategy: FoodAid/EducationImpactStrategy
CR-12	Admins add or rename skills and categories without a developer, but cannot delete one that is in use.	CRUD: GenericDAO, LookupController.isInUse()
CR-13	Passwords are stored only as hashes.	PersonService.hashPassword()

5. Non-Functional Requirements

ID	Quality	Requirement
NFR-1	Maintainability	Each class has one responsibility and depends on interfaces or abstract classes where behaviour varies, so a change is made in one place.
NFR-2	Extensibility (open/closed)	A new payment method, delivery channel, impact measure, document extra, ticket extra, donation type, event type or observer is added by writing one new class, without editing existing ones.
NFR-3	Usability	Menus are numbered; every invalid entry (non-number, bad date, unknown id) shows a clear message and asks again instead of crashing.
NFR-4	Data consistency	All services and controllers share one data store instance (Singleton), so every screen sees the same data.
NFR-5	Data integrity	Invalid donations and events are rejected; stock, capacity and in-use lookup rows are protected; e-mails are unique.

ID	Quality	Requirement
NFR-6	Security	Passwords are stored as SHA-256 hashes; payment card numbers are masked.
NFR-7	Portability	Runs on any operating system with Java 17 and Maven; no external services are required.
NFR-8	Performance	Every operation completes instantly for the expected data size (hundreds of records held in memory).

6. System Design

6.1 Architecture - Model View Controller

The code is organized in three layers. The user only talks to a **View**; the **Controller** reads the user's choices from its view, calls the **Model** (services and domain classes) and sends the results back to the view.

Layer	Packages	Responsibility
Model	com.nonprofit.model.* - persistence, lookup, people, notification (+ channels), observer, donation (+ factory), payment, document, event (+ factory, ticket), volunteer, beneficiary	Business data and rules: services, domain classes, patterns and the data store. Never prints.
View	com.nonprofit.view - View (abstract) and MainView, PersonView, DonationView, EventView, VolunteerView, BeneficiaryView, LookupView	The only layer that reads input and prints output. View holds the shared input and output helpers.
Controller	com.nonprofit.controller - MainController and Person, Donation, Event, Volunteer, Beneficiary and Lookup controllers	One controller per menu. MainController is also the composition root: it creates the services, plugs in the notification strategies and subscribes the observers.
Entry point	com.nonprofit.Main	Creates MainView and MainController and starts the application.

This separation follows the course rule "never put output statements inside core classes": for example, a receipt returns its text through `render()` and the view prints it. The same model could therefore be reused behind a web or desktop interface in Phase 2.

Design quality checklist

The class design was checked against the course's list of common class-diagram mistakes:

Rule	How the design meets it
No empty classes	All 94 classes have attributes or operations; enumerations list their values.
No standalone classes	Every class, including each enumeration, takes part in at least one of the 167 relationships.
No inheritance cycles	All hierarchies are trees.
No single inheritance	Every parent has at least two children and every interface at least two implementations, e.g. Person (3), Event (3), EventCreator (3), Document (3), DocumentDecorator (3), View (7), Observer (6), Subject (3), Identifiable (8).
Use interfaces	Identifiable, Subject, Observer, NotificationStrategy, PaymentStrategy, ImpactEvaluationStrategy and Ticket are interfaces; clients depend on them rather than on concrete classes.
Balanced relationships	The diagrams use association, aggregation, composition, generalization, realization and dependency, with multiplicities on every association.
No system class	There is no all-knowing "System" class; work is split across focused services, controllers and domain classes.
No output in core classes	Only the View layer prints.

6.2 Class Diagrams

Seven diagrams are provided: the full system and one focused view per Phase 1 grading item. All of them are generated from one shared PlantUML model written from the code, so they always agree with each other and with the implementation. High-resolution PNG and SVG copies and the PlantUML source are in the "Class Diagrams" folder of the submission.

Notation: + public, - private, # protected; underlined members are static; italic names and operations are abstract; filled diamond = composition, hollow diamond = aggregation, solid arrow = association, dashed arrow = dependency, hollow triangle = generalization (solid) or realization (dashed).

Figure	Diagram	Phase 1 grading item
Figure 1	Full class diagram - all 94 classes, interfaces and enumerations with their 167 relationships	Class diagram
Figure 2	Payment, notification channel and impact evaluation	Strategy pattern
Figure 3	Donation, event and distribution notifications	Observer pattern
Figure 4	Receipts, certificates and tickets	Decorator pattern
Figure 5	Creating donations and events	Factory pattern (Factory Method)
Figure 6	The shared data store	Singleton pattern
Figure 7	Application layers and lookup tables	MVC and CRUD lookup tables

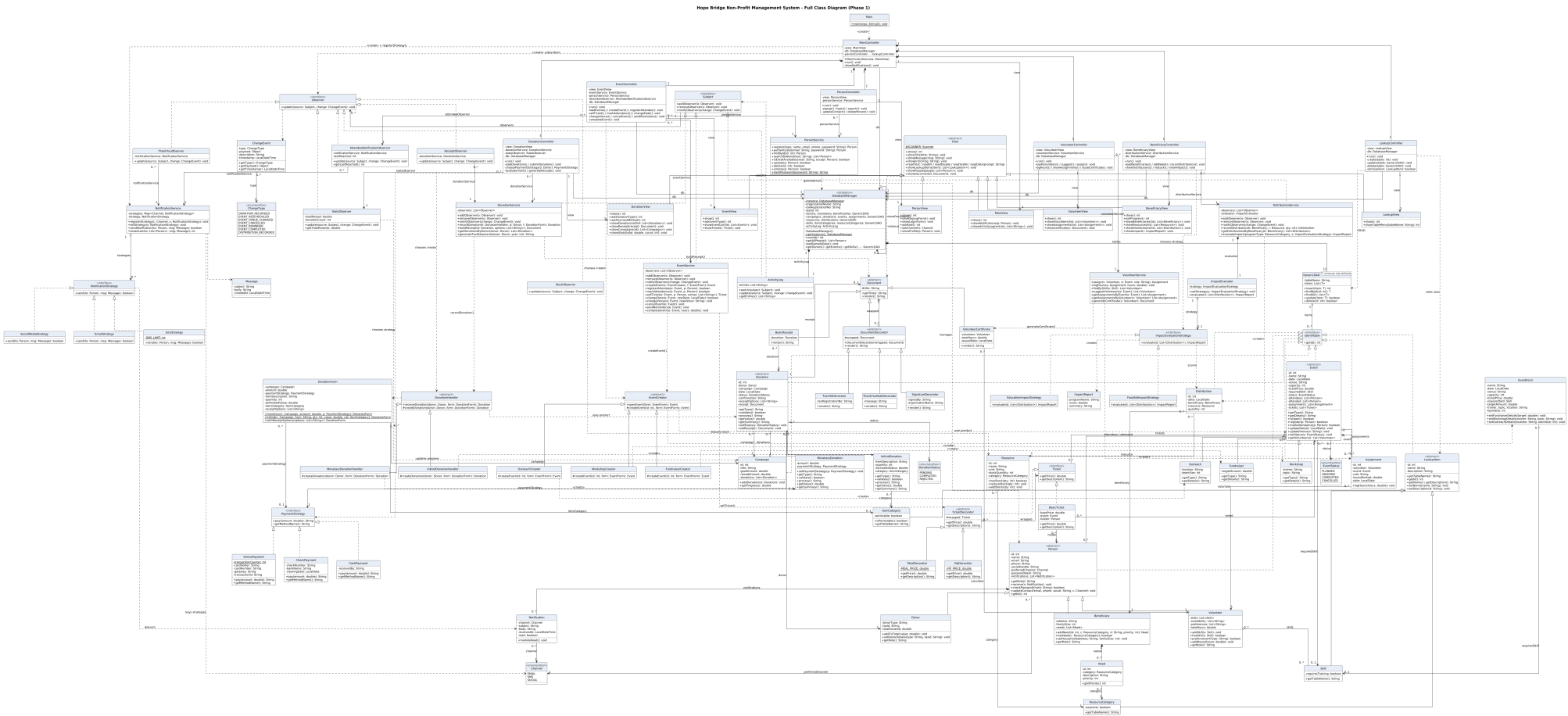


Figure 1: Full class diagram of the system

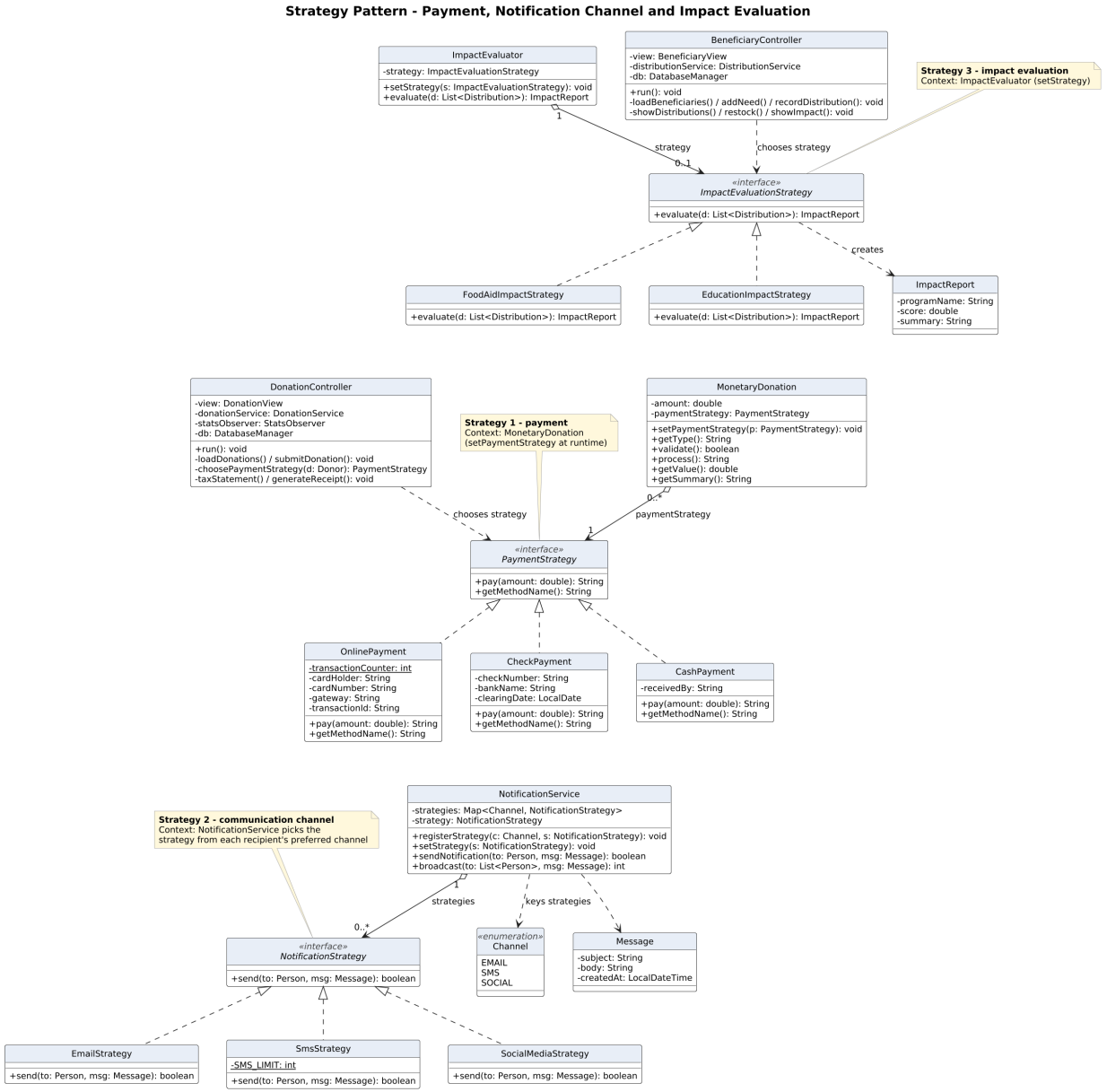


Figure 2: Strategy pattern - payment, notification channel and impact evaluation

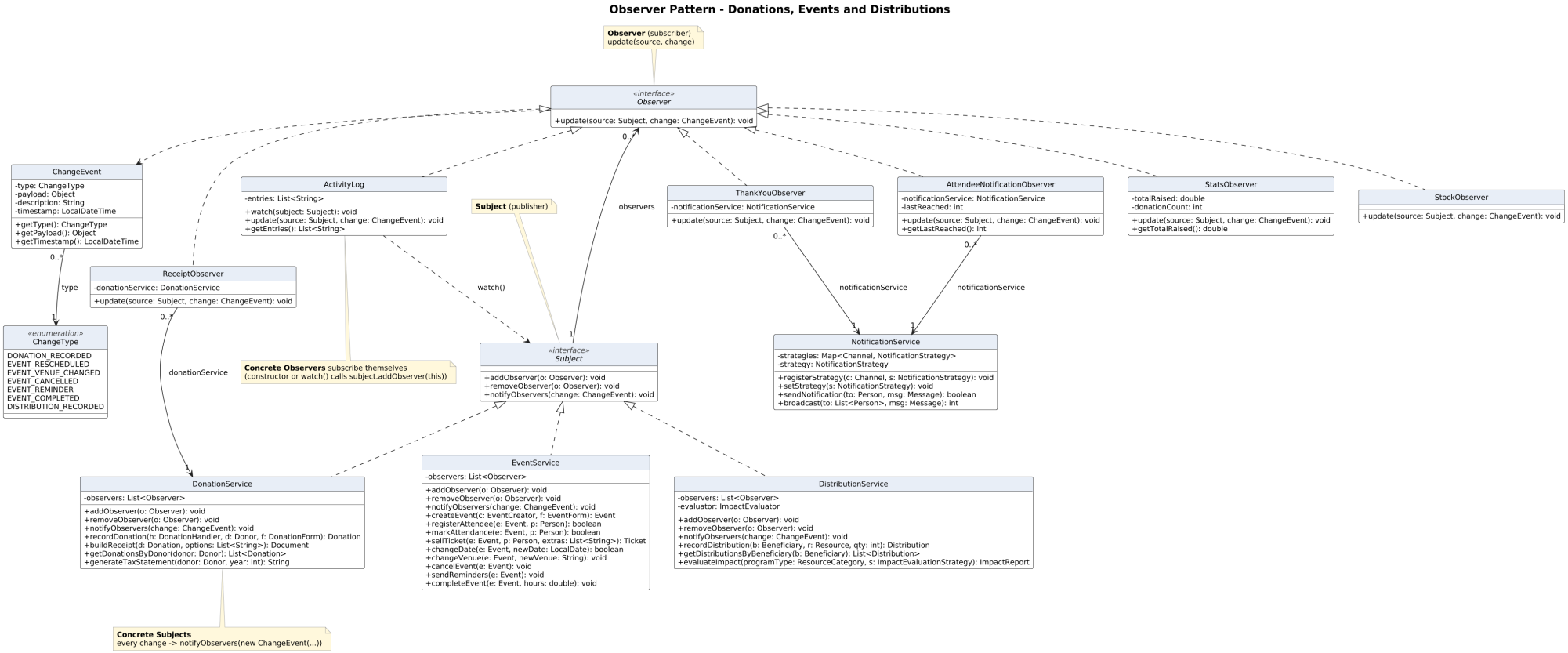


Figure 3: Observer pattern - donations, events and distributions

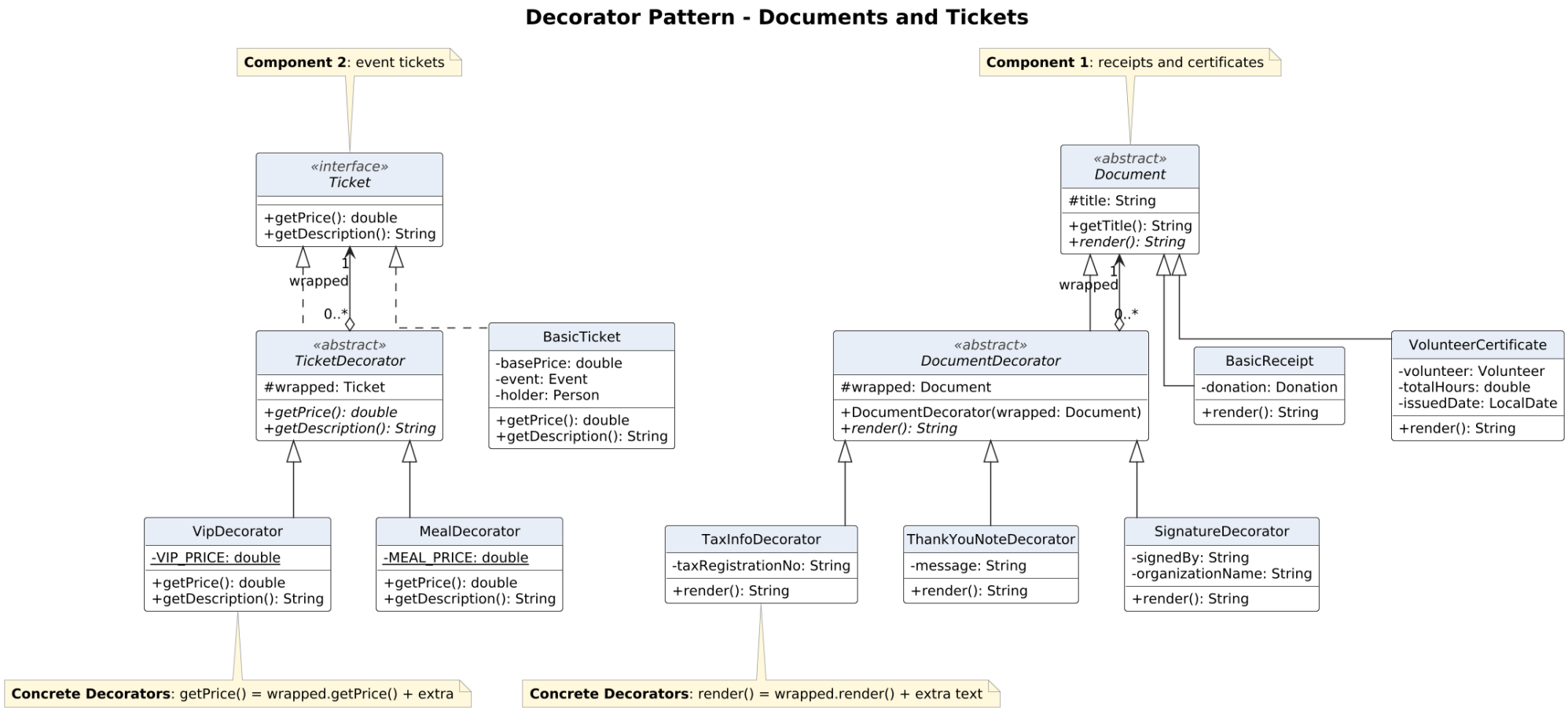


Figure 4: Decorator pattern - documents and tickets

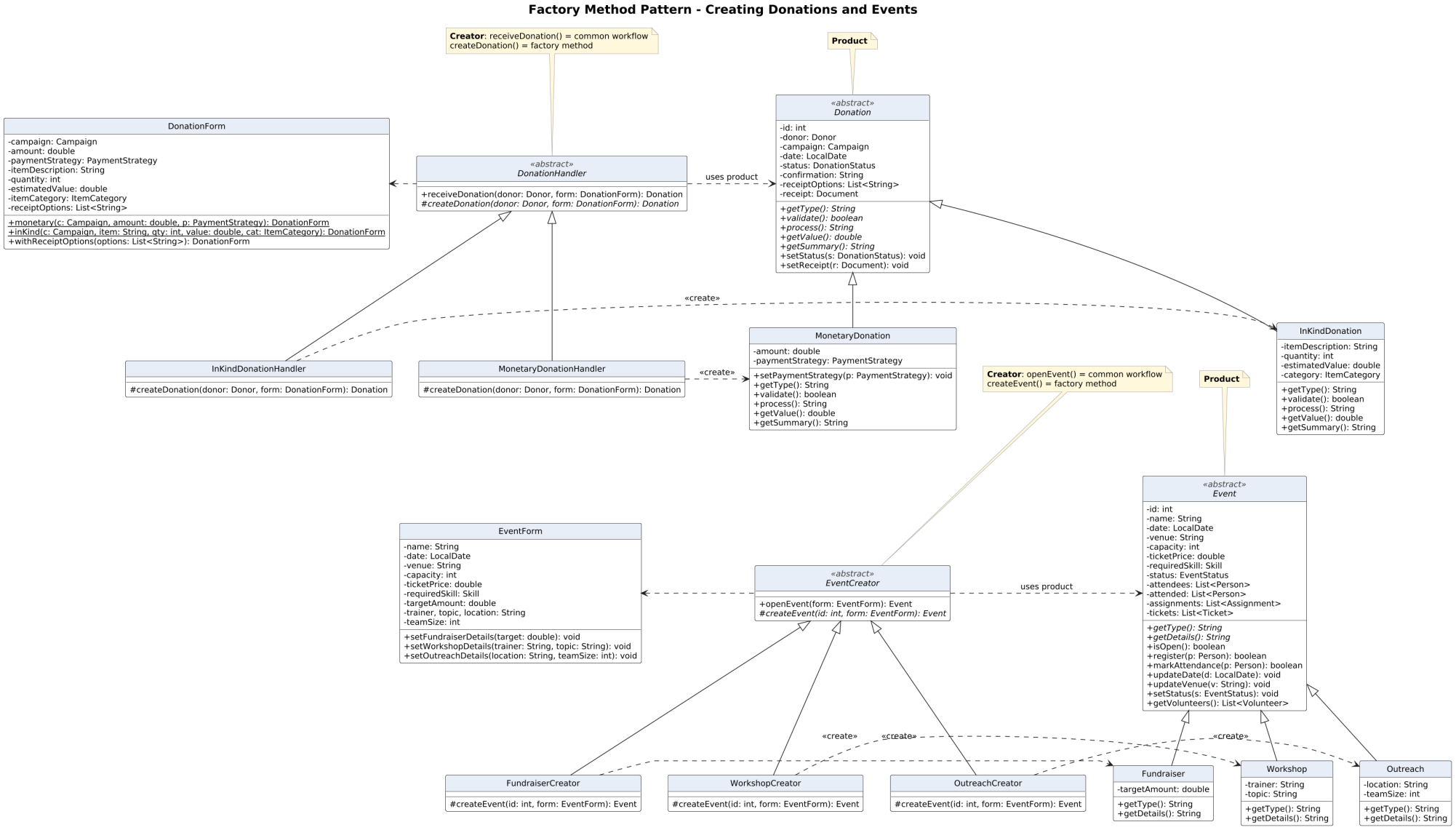


Figure 5: Factory Method pattern - creating donations and events

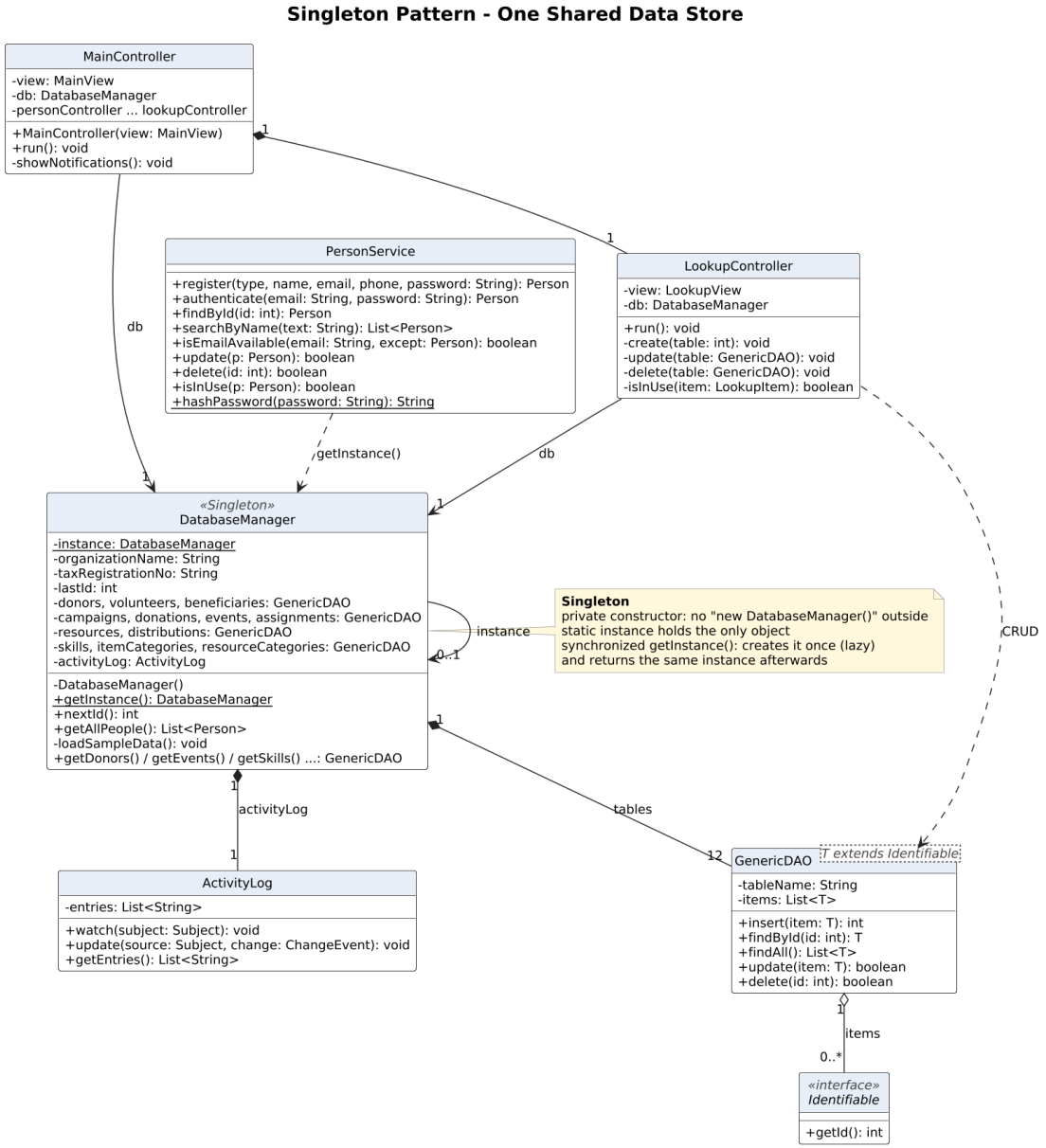


Figure 6: Singleton pattern - the shared data store

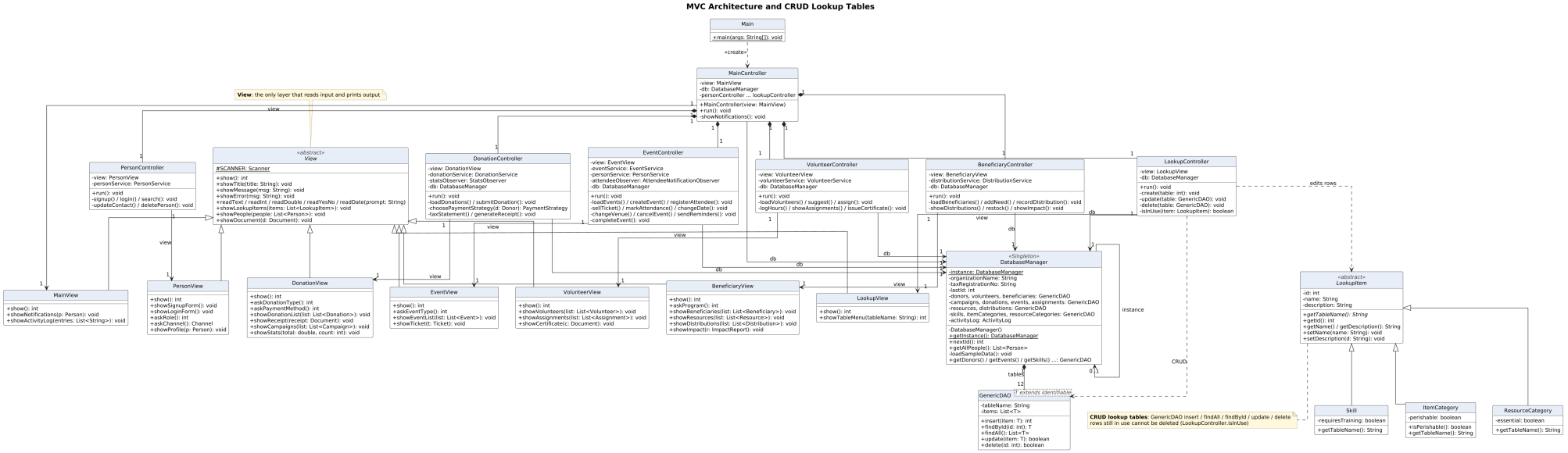


Figure 7: MVC architecture and CRUD lookup tables

6.3 Design Patterns Applied

6.3.1 Strategy (Figure 2)

Problem. Several behaviours have many interchangeable variants: how a donation is paid, how a message is delivered, and how a program's impact is measured. An if/else on the variant would have to be edited every time a variant is added, breaking the open/closed principle.

Solution. Each variant is a class behind a common interface, and a context object holds a reference to one strategy and delegates to it. The pattern is applied three times:

Use	Strategy interface	Concrete strategies	Context (how the strategy is chosen)
Payment	PaymentStrategy	OnlinePayment, CheckPayment, CashPayment	MonetaryDonation - setPaymentStrategy(); the donor chooses in DonationController
Communication channel	NotificationStrategy	EmailStrategy, SmsStrategy, SocialMediaStrategy	NotificationService - picks the strategy from each recipient's preferredChannel; setStrategy() can force one
Impact evaluation	ImpactEvaluationStrategy	FoodAidImpactStrategy, EducationImpactStrategy	ImpactEvaluator - setStrategy(); the program type is chosen in BeneficiaryController

```
// NotificationService (Context)
public boolean sendNotification(Person to, Message msg) {
    NotificationStrategy chosen = strategy != null
            ? strategy : strategies.get(to.getPreferredChannel());
    if (chosen != null && chosen.send(to, msg)) {
        return true;
    }
    ...  // fall back to e-mail
}
```

Benefit. A WhatsApp channel, a mobile-wallet payment or a health-program impact measure is one new class plus one registration line; the contexts do not change.

6.3.2 Observer (Figure 3)

Problem. When a donation is recorded, an event changes or resources are distributed, many independent reactions must follow (receipt, thank-you, totals, attendee messages, stock, audit log). If the services called each reaction directly they would be tightly coupled to all of them.

Solution. DonationService, EventService and DistributionService implement `Subject` and keep a list of `Observers`. Every state change creates a `ChangeEvent` (type, payload, description, timestamp) and calls `notifyObservers(change)`, which calls `update(source, change)` on each observer. Observers subscribe themselves in their constructor (or through `watch()`), as in the Lab 4 examples.

Role	Class	In the code
Subject	Subject (interface)	addObserver(), removeObserver(), notifyObservers(change)
Observer	Observer (interface)	update(source, change)
Concrete subjects	DonationService, EventService, DistributionService	recordDonation(); changeDate(), changeVenue(), cancelEvent(), sendReminders(), completeEvent(); recordDistribution()
Concrete observers	ReceiptObserver, ThankYouObserver, StatsObserver	Attach the decorated receipt; thank the donor; update donor, campaign and organization totals

Role	Class	In the code
Concrete observers	AttendeeNotificationObserver, StockObserver, ActivityLog	Message attendees and volunteers; reduce stock; record every change

```
// EventService (Concrete Subject)
public void cancelEvent(Event event) {
    event.setStatus(EventStatus.CANCELLED);
    notifyObservers(new ChangeEvent(ChangeType.EVENT_CANCELLED, event,
            "'" + event.getName() + "' was cancelled."));
}
```

Benefit. Subjects only know the Observer interface, so a new reaction (for example a Phase 2 SMS gateway or a dashboard) is one new observer; the services do not change.

6.3.3 Decorator (Figure 4)

Problem. Receipts, certificates and tickets need optional extras in any combination. A subclass for every combination would cause a class explosion.

Solution. Each family has a component type, a plain concrete component and an abstract decorator that is a component and wraps one (`wrapped`). Concrete decorators return the wrapped result plus their own part.

Family	Component	Concrete components	Decorator	Concrete decorators
Documents	Document (render())	BasicReceipt, VolunteerCertificate	DocumentDecorator	TaxInfoDecorator, ThankYouNoteDecorator, SignatureDecorator
Tickets	Ticket (getPrice(), getDescription())	BasicTicket	TicketDecorator	VipDecorator (+250 EGP), MealDecorator (+150 EGP)

```
// DonationService.buildReceipt()
Document receipt = new BasicReceipt(donation);
for (String option : options) {
    if (option.equals("TAX")) {
        receipt = new TaxInfoDecorator(receipt, db.getTaxRegistrationNo());
    } else if (option.equals("THANK_YOU")) {
        receipt = new ThankYouNoteDecorator(receipt, "Thank you, ...");
    }
}
return new SignatureDecorator(receipt, "Finance Manager",
db.getOrganizationName());
```

Benefit. With n decorators and m components only n + m classes are needed; the same document decorators serve receipts and certificates.

6.3.4 Factory Method (Figure 5)

Problem. Every donation and every event follows a common workflow, but the concrete class differs (money or goods; fundraiser, workshop or outreach). Creating them with `new` inside the workflow would tie it to every concrete type.

Solution. An abstract creator implements the common workflow and declares a protected abstract factory method; each concrete creator decides which product to instantiate. Product and creator hierarchies are parallel, as in the Pizza Store example.

Role	Donations	Events
Creator	DonationHandler: receiveDonation() workflow (create, validate, process,	EventCreator: openEvent() workflow (validate, create, save)

Role	Donations	Events
	save)	
Factory method	#createDonation(donor, form)	#createEvent(id, form)
Concrete creators	MonetaryDonationHandler, InKindDonationHandler	FundraiserCreator, WorkshopCreator, OutreachCreator
Product	Donation	Event
Concrete products	MonetaryDonation, InKindDonation	Fundraiser, Workshop, Outreach

```
public abstract class EventCreator {
    public Event openEvent(EventForm form) {
        if (/* name, date, capacity or price invalid */) {
            return null;
        }
        Event event = createEvent(DatabaseManager.getInstance().nextId(), form);
        DatabaseManager.getInstance().getEvents().insert(event);
        return event;
    }
    protected abstract Event createEvent(int id, EventForm form);
}
```

Benefit. A new donation or event type is one new product and one new creator; the workflow is not touched. This replaces the static "create(type)" simple factory of the first draft, which needed an if/else edit for every new type.

6.3.5 Singleton (Figure 6)

Problem. Every service and controller must work on the same people, donations, events and lookup tables. Two data stores would hold different data and the system would become inconsistent.

Solution. `DatabaseManager` has a private constructor, a static `instance` and a static synchronized `getInstance()` that creates the instance on first use (lazy initialization) and returns the same object afterwards. It holds one `GenericDAO<T>` per kind of record.

```
private static DatabaseManager instance;
private DatabaseManager() { ... loadSampleData(); }
public static synchronized DatabaseManager getInstance() {
    if (instance == null) {
        instance = new DatabaseManager();
    }
    return instance;
}
```

Benefit. One global access point for data; in Phase 2 the in-memory DAOs can be backed by a real database behind the same class without changing its clients.

6.4 CRUD Lookup Tables

Reference values that change over time are not hard-coded. They are rows of lookup tables that the administrator maintains from the "Lookup tables (admin)" menu.

Lookup table	Row class	Extra field	Used by	Sample rows
Volunteer skills	Skill	requiresTraining	Volunteer.skills, Event.requiredSkill	First Aid, Teaching, Driving
Item categories	ItemCategory	perishable	InKindDonation.category	Food, Clothes, Medicine
Resource categories	ResourceCategory	essential	Resource.category, Need.category, impact evaluation	Food Aid, Education, Medical

All three extend the abstract `LookupItem` (id, name, description) and are stored in the generic `GenericDAO<T extends Identifiable>`, which provides `insert()`, `findAll()`, `findById()`, `update()` and `delete()`. `LookupController.isInUse()` blocks the deletion of a row that an event, volunteer, donation, resource or need still references.

6.5 Main Design Decisions

The design merges the team's first draft with the course guidelines. Where the draft and the guidelines differed, the following choices were made:

First draft	Final design	Reason
Static DonationFactory / EventFactory with create(type)	Factory Method: DonationHandler and EventCreator hierarchies	The labs present the simple factory only as a first step; Factory Method keeps creation open for extension.
OnlineDonation / CheckDonation subclasses	MonetaryDonation with a PaymentStrategy	Payment is behaviour that varies at runtime, so it is a Strategy; subclasses per method would mix two concerns.
Categories stored as plain Strings	Lookup tables Skill, ItemCategory, ResourceCategory	Editable at runtime (CRUD) and protected from deletion while in use.
Receipt interface for receipts only	Document component for receipts and certificates	One set of decorators (tax, thank-you, signature) serves both kinds of document.
View interface	Abstract View class	All views share the same input and output code, which an abstract class can hold.
DatabaseManager with a JDBC connection	DatabaseManager with in-memory GenericDAOs	Phase 1 needs no database server; Phase 2 can add one behind the same class.

7. Work Distribution

Each member owns a vertical slice of the system and the pattern used in it. Every one of the 94 classes is assigned to exactly one member.

Member	ID	Area	Patterns	Classes
Saifeldin Tamer ElSayes	22P0233	Foundation, people and lookup tables: persistence, lookup, people, MVC entry	Singleton, CRUD lookup tables, MVC	Main, MainController, MainView, View, DatabaseManager, GenericDAO, Identifiable, Person, Donor, Volunteer, Beneficiary, PersonService, PersonView, PersonController, LookupItem, Skill, ItemCategory, ResourceCategory, LookupView, LookupController
Ahmed Khaled Hamdy	22P0024	Observer and notification core: observer, notification	Observer, Strategy (context)	Subject, Observer, ChangeEvent, ChangeType, ReceiptObserver, ThankYouObserver, StatsObserver, AttendeeNotificationObserver, StockObserver, ActivityLog, NotificationService, NotificationStrategy, Channel, Message, Notification
Mohamed Emad Eldeen	22P0065	Donations and notification channels: donation, payment, notification.channels	Factory Method, Strategy	DonationService, DonationHandler, MonetaryDonationHandler, InKindDonationHandler, Donation, MonetaryDonation, InKindDonation, DonationStatus, DonationForm, Campaign, PaymentStrategy, OnlinePayment, CheckPayment, CashPayment, DonationView, DonationController, EmailStrategy, SmsStrategy, SocialMediaStrategy
Seif Eldeen Ibrahim	22P0023	Events and volunteer logic: event, volunteer	Factory Method, Observer (subject)	EventService, EventCreator, FundraiserCreator, WorkshopCreator, OutreachCreator, Event, Fundraiser, Workshop, Outreach, EventStatus, EventForm, EventView, EventController, VolunteerService, Assignment, VolunteerController
Yousef Sameh Fathi	22P0010	Decorators and volunteer output: document, event.ticket	Decorator	Document, BasicReceipt, DocumentDecorator, TaxInfoDecorator, ThankYouNoteDecorator, SignatureDecorator, Ticket, BasicTicket, TicketDecorator, VipDecorator, MealDecorator, VolunteerView, VolunteerCertificate
Mohamed Khaled Mohamed Anwar	22P0041	Beneficiaries and impact evaluation: beneficiary	Strategy, Observer (subject)	DistributionService, Distribution, Resource, Need, ImpactEvaluator, ImpactEvaluationStrategy, FoodAidImpactStrategy, EducationImpactStrategy, ImpactReport, BeneficiaryView, BeneficiaryController

8. Traceability to the Phase 1 Grading Criteria

Grading item	Marks	Diagram	Classes	Source location
Class diagram	1	Figure 1 (and all figures)	All 94 classes, interfaces and enumerations	Class Diagrams/
Strategy pattern	0.5	Figure 2	PaymentStrategy, NotificationStrategy, ImpactEvaluationStrategy and their concrete strategies; MonetaryDonation, NotificationService, ImpactEvaluator	model/payment, model/notification, model/beneficiary
Observer pattern	0.5	Figure 3	Subject, Observer, ChangeEvent; Donation/Event/DistributionService; 6 concrete observers	model/observer and the three services
Decorator pattern	0.5	Figure 4	Document + DocumentDecorator family; Ticket + TicketDecorator family	model/document, model/event/ticket
Factory pattern	0.5	Figure 5	DonationHandler and EventCreator hierarchies with their products	model/donation/factory, model/event/factory
Singleton pattern	0.5	Figure 6	DatabaseManager (with GenericDAO)	model/persistence
MVC	0.5	Figure 7	Main, 7 controllers, View and 7 views	controller, view
Client requirements	0.5	-	Section 4 (CR-01 to CR-13)	-
CRUD lookup tables	0.5	Figure 7	LookupItem, Skill, ItemCategory, ResourceCategory, GenericDAO, LookupController, LookupView	model/lookup, model/persistence

Source paths are relative to `2-Code/NonProfitSystem/src/main/java/com/nonprofit/`.

9. Running the System

9.1 Build and Run

From the `NonProfitSystem` folder (requires JDK 17+ and Maven):

```
mvn compile
java -cp target/classes com.nonprofit.Main
```

Alternatively, open the folder as a Maven project in NetBeans or IntelliJ and run `com.nonprofit.Main`.

9.2 Sample Data

The data store is loaded with sample data at start-up (every sample account's password is pass123). Ids are assigned in this order:

Ids	Records
1 - 3	Skills: First Aid, Teaching, Driving
4 - 6	Item categories: Food, Clothes, Medicine
7 - 9	Resource categories: Food Aid, Education, Medical

Ids	Records
10 - 11	Donors: Omar Hassan (individual, e-mail), Mona Adel (corporate, SMS)
12 - 13	Volunteers: Laila Fathy (Teaching, First Aid; social media; prefers workshops), Karim Nabil (Driving; SMS; prefers outreach)
14 - 18	Beneficiaries: Amal Saeed (14, family of 5, needs food aid and education), Hany Mostafa (17, family of 3, needs medical supplies)
19	Campaign: Winter Blankets 2026 (goal 50,000 EGP)
20 - 22	Events: Winter Charity Gala (fundraiser, ticket 500 EGP), Literacy Workshop (needs Teaching), Food Convoy (outreach, needs Driving)
23 - 25	Resources: Food box (120), School kit (60), First-aid kit (40)

9.3 Demonstration Scenario

1. **Donations > Make a donation:** donor 10, campaign 19, money, 500 EGP, online card, tax info and thank-you note. The receipt (Decorator) and a thank-you e-mail (Observer + Strategy) are produced; repeat as donor 11 with 20 blankets (Factory Method creates an InKindDonation; the thank-you arrives by SMS).
2. **Donations > Campaign progress** and **Donor tax statement** for donor 10.
3. **Events:** register person 14 for event 21, sell person 10 a VIP + dinner ticket for event 20 (900 EGP), mark person 14 as attended, change the date of event 21 and send reminders - attendees are notified (Observer).
4. **Volunteers:** suggest volunteers for event 21, try to assign volunteer 13 (refused: no Teaching skill), assign volunteer 12; then **Events > Complete** event 21 with 4 hours and **Volunteers > Issue a certificate** for volunteer 12.
5. **Beneficiaries:** give 10 food boxes and 3 school kits to beneficiary 14 (stock drops automatically), try 500 food boxes for 17 (refused), then evaluate the Food Aid program with the food-aid strategy and Education with the education strategy.
6. **People:** sign up a new volunteer, log in, search, try to delete donor 10 (refused: has donations). **Lookup tables:** add a resource category and try to delete Food Aid (refused: in use).
7. **View a person's notifications** for persons 10, 11, 12 and 14 to see each channel, then **View activity log**.